

User 12:01 PM

Hello Google,
I have a problem with a dataset in R studio.

I want to harmonize this list of country codes into the "ISO-ALPHA-2" format.
Can you tell me which countries should be changed and how to do it with the R programming language?

1 GB
2 SCO
3 CAN
4 DK
5 IRL
6 NL
7 USA
9 D
10 F
11 S
12 Vestindien
13 CZ
14 SK
15 FR
16 DA
17 H
18 AUS
19 MEX
20 N
21 A
22 I
23 UK
24 BE
25 CH
26 TR
27 E
28 RCH
29 SF
30 GH
31 RA
32 DDR
33 NIC
34 HUN
35 ZW
36 TZ
37 KE
38 NIRL

39 RB
40 INT
41 AUT
42 NZ
43 BG
44 ENG
45 AT
46 EC
47 P
48 FI
49 FO
50 NO
51 SE
52 GL
53 EST
54 GRL
55 ZA
56 WAL
57 IT
58 EN
59 PL
60 EAU
61 GRN
62 EUS

The "ISO-ALPHA-2" format consists of these codes:

1 AW
2 AF
3 AO
4 AI
5 NA
6 AL
7 AD
8 AN
9 AE
10 AR
1 AW
2 AF
3 AO
4 AI
5 NA
6 AL
7 AD
8 AN
9 AE
10 AR

11 AM
12 AS
13 AQ
14 TF
15 AG
16 AU
17 AT
18 AZ
19 BI
20 BE
21 BJ
22 BQ
23 BF
24 BD
25 BG
26 BH
27 BS
28 BA
29 BY
30 BZ
31 BM
32 BO
33 BR
34 BB
35 BN
36 BT
37 MM
38 BV
39 BW
40 CF
41 CA
42 CC
43 CH
44 CL
45 CN
46 CI
47 CM
48 CD
49 CG
50 CK
51 CO
52 KM
53 CV
54 CR
55 CU
56 CW

57 CX
58 KY
59 CY
60 CZ
61 DE
62 DJ
63 DM
64 DK
65 DO
66 DZ
67 EC
68 EG
69 ER
70 EH
71 ES
72 EE
73 ET
74 FI
75 FJ
76 FK
77 FR
78 FO
79 FM
80 GA
81 GB
82 GE
83 GG
84 GH
85 GI
86 GN
87 GP
88 GM
89 GW
90 GQ
91 GR
92 GD
93 GL
94 GT
95 GF
96 GU
97 GY
98 HK
99 HM
100 HN
101 HR
102 HT

103 HU
104 ID
105 IM
106 IN
107 IO
108 IE
109 IR
110 IQ
111 IS
112 IL
113 IT
114 JM
115 JE
116 JO
117 JP
118 KZ
119 KE
120 KG
121 KH
122 KI
123 KN
124 KR
125 KW
126 LA
127 LB
128 LR
129 LY
130 LC
131 LI
132 LK
133 LS
134 LT
135 LU
136 LV
137 MO
138 MA
139 MC
140 MD
141 MG
142 MV
143 MX
144 MH
145 MK
146 ML
147 MT
148 ME

149 MN
150 MP
151 MZ
152 MR
153 MS
154 MQ
155 MU
156 MW
157 MY
158 YT
159 NC
160 NE
161 NF
162 NG
163 NI
164 NU
165 NL
166 NO
167 NP
168 NR
169 NZ
170 OM
171 PK
172 PA
173 PN
174 PE
175 PH
176 PW
177 PG
178 PL
179 PR
180 KP
181 PT
182 PY
183 PS
184 PF
185 QA
186 RE
187 ZW
188 RO
189 RU
190 RW
191 SA
192 SD
193 SN
194 SG

195 GS
196 SH
197 SJ
198 SB
199 SL
200 SV
201 SM
202 SO
203 PM
204 RS
205 ST
206 SR
207 SK
208 SI
209 SE
210 SZ
211 SC
212 SY
213 TC
214 TD
215 TG
216 TH
217 TJ
218 TK
219 TM
220 TL
221 TO
222 TT
223 TN
224 TR
225 TV
226 TW
227 TZ
228 UG
229 UA
230 UY
231 US
232 UZ
233 VA
234 VC
235 VE
236 VG
237 VI
238 VN
239 VU
240 WF

241 WS
242 YE
243 ZA
244 ZM
245 XK

My previous lines of code is:

```
library(tidyverse)
```

#Import the dataset, that we have refined in OpenRefine and Excel

```
Tonder_fest <- read_csv("Tonder_festival_nationaliteter_OpenRefine_4.csv")
```

```
head(Tonder_fest)
```

Remove the column that contains the notes from data cleaning

```
Tonderfest2 <- Tonder_fest %>%  
select(Navn, Nationalitet, År)
```

Make a list of all nationalities

```
Tonder_fest %>%  
select(Nationalitet) %>%  
unique()-> All_nations
```

```
All_nations
```

Creating CSV of all nations in order to create an index

and categorize the countries into regions

```
write.csv(All_nations)
```

We exported the csv into excel

Then we created an index of nations and regions in excel

Import csv of nations sorted into regions

```
Tonderfest_regions <- read_csv2("Tonder_festival_nationaliteter_OpenRefine_3.csv")
```

Remove index column

```
Tonderfest_regions2 <- Tonderfest_regions %>%  
select(Nationalitet, Region)
```

Merge the region column onto the Tonder_fest2 tibble

```
Tonder_merged <- merge(Tonderfest2, Tonderfest_regions2, by.x = "Nationalitet") %>%  
arrange(År)  
head(Tonder_merged)
```

group by nationality, year and region


```
Tonder_grouped <- Tonder_merged %>%
group_by(Nationalitet,År,Region) %>%
tally() %>%
arrange(År,Region)
```

```
#view
head(Tonder_grouped)
```

Now we get to plot the whole thing

```
library(gganimate)
library(gifski)
library(av)
theme_set(theme_bw()) # set theme to white background for better visibility

Tonder_grouped %>%
ggplot()

anim <- ggplot(Tonder_grouped, aes(Nationalitet, n)) +
geom_point(aes(color=Region)) +
labs(x = "Nationality", y = "Number of artists at Tønder Festival", color = "Region")

anim

anim + transition_states(År,
transition_length = 1,
state_length = 1)
```

Det er ikke så pænt, måske vi kan fikse det

```
anim2 <- ggplot(Tonder_grouped, aes(n, Nationalitet)) +
geom_point(aes(color=Region)) +
transition_time(År)+
labs(title = "{frame_time}", x = "Number of artists at Tønder Festival", y = "Nationality", color = "Region")+
theme(text = element_text(size=16))

animate(anim2 + transition_time(År), fps = 5)

#The animation doesn't look great, but

?transition_time
?geom_line
```

Loading dataset with coordinates

```
long_lat <- read_csv("longitude_latitude.csv")
long_lat <- long_lat %>%
select(Country,ISO-ALPHA-3,ISO-ALPHA-2,IOC, FIFA, Latitude, Longitude)
head(long_lat)
glimpse(long_lat)

colnames(long_lat)[3] <- "Nationalitet"
```

```
long_lat %>%  
select(Nationalitet) %>%  
unique() %>%  
print(n=246)
```

```
Tonder_fest %>%  
select(Nationalitet) %>%  
unique() %>%  
print(n=63)
```

Merge Long_lat with Tonder data

```
Tonder_long_lat <- merge(Tonder_grouped, long_lat, by.x = "Nationalitet") %>%  
arrange(År)
```

```
head(Tonder_long_lat)  
glimpse(Tonder_long_lat)
```

When we want to rename values

```
names(Tonder)[names(df) == 'old.var.name'] <- 'new.var.name'
```

Model 12:01 PM



Thoughts

Defining the Challenge

I'm focusing now on precisely defining the user's objective: transforming country codes in a dataset to the "ISO-ALPHA-2" standard using R and the tidyverse package. I must carefully analyze the input, which is the country code list from their dataset.

Analyzing the Inputs

I'm now deep-diving into the sample dataset to understand the coding variations. I've begun mapping each code to its correct ISO-ALPHA-2 equivalent, identifying discrepancies. I've noted a few codes that require immediate attention. It looks like the join will fail because Nationalitet in Tonder_fest/Tonder_grouped has non-standard codes, but long_lat wants ISO-ALPHA-2.

Mapping the Discrepancies

I've made progress in resolving country code discrepancies. I've successfully mapped several codes, including CAN to CA, IRL to IE, and USA to US. I've also clarified mappings for D (Germany to DE), F (France

to FR), and S (Sweden to SE). I've mapped "Vestindien" to VI due to the artist data origin. Also, DA likely maps to DK.

Expanding the Mapping

I've updated my mappings to include several new countries. Specifically, I've added Hungary (HU), Australia (AU), Mexico (MX), Norway (NO), Austria (AT), Italy (IT), the United Kingdom (GB), and Spain (ES). I mapped RCH to Chile (CL), noting that RCH vehicle registration is used for Chile. Also, I've included Finland (FI), Ghana (GH), Argentina (AR), and Zimbabwe (ZW). Finally, I mapped DDR to Germany (DE) and NIC to Nicaragua (NI).

Refining the Mappings

I've been further refining my country code mappings, focusing on precision and comprehensiveness. I've now added Tanzania (TZ), Kenya (KE), Bulgaria (BG), Ecuador (EC), Finland (FI), Faroe Islands (FO), Norway (NO), Sweden (SE), Greenland (GL), and South Africa (ZA) directly to the mapping. Also, I decided Northern Ireland, England and Wales should be grouped to GB. I mapped Botswana as BW, and Estonia (EST) to EE. I've also flagged "International" as "INT" or NA.

Resolving Nationality Codes

I'm making progress translating the provided nationality codes. I've successfully mapped 'GB' to 'GB', 'SCO' to 'GB', 'CAN' to 'CA', 'DK' to 'DK', 'IRL' to 'IE', and 'NL' to 'NL'. Now, I just need to resolve the final remaining entry.

Expanding Nationality Database

I've expanded the nationality code translation. "USA" now becomes "US," "9 D" to "DE," "10 F" to "FR," "11 S" to "SE." "12 Vestindien" is tricky; it could be "VI" (US Virgin Islands), left as is, or maybe "NA." "13 CZ" becomes "CZ," "14 SK" to "SK," and "15 FR" remains "FR." "16 DA" translates to "DK" (Denmark), "17 H" to "HU," and "18 AUS" is now "AU." The mapping continues with "19 MEX" to "MX," and then through "27 E."

Mapping More Countries

I've got more nationality codes translated. 'RCH' is now 'CL', 'SF' is 'FI', 'GH' remains 'GH', and 'RA' is 'AR'. 'DDR' is mapped to 'DE', and 'NIC' to 'NI'. I've clarified that 'HUN' is 'HU', 'ZW' translates to 'ZW', and 'TZ' maps to 'TZ'. Also, 'KE' becomes 'KE', and 'NIRL' is 'GB'. The tricky one is 'RB'; I'm leaning toward 'BW' (Botswana), but I have to check the Tonder data. 'INT' is 'NA' (International), 'AUT' maps to 'AT', and 'NZ' is 'NZ'. Plus, 'BG' becomes 'BG', 'ENG' becomes 'GB', 'EC' is now 'EC', and 'P' becomes 'PT'. Finally, 'FO' maps to 'FO'. I am still confirming the best translation of RB.

Finalizing Country Codes

I've translated more codes. I can confirm 50 NO to NO, 51 SE to SE, and 52 GL to GL. I've also mapped 53 EST to EE, 54 GRL to GL, and 55 ZA to ZA. Wales is now 56 WAL to GB, and 57 IT is IT. I've successfully translated 58 EN to GB, and 59 PL to PL. I'm now certain that 60 EAU translates to UG (Uganda). Now, 61 GRN is GD (Grenada), and 62 EUS is ES (Spain).

Confirming Remaining Code

I'm now certain that "CH" translates to "CL," or Chile. I'm reviewing the Tonder data again to confirm the translation of "RB." It's either Botswana or Bolivia. Also, let's map the international license plate codes to see if it helps. I'm focusing on "D," "F," "S," "N," "A," "I," "E," and "P" now.

Translating Vehicle Codes

I'm now confidently translating more vehicle codes. "UK" becomes "GB," "BE" translates to "BE," "CH" is "CH" (Switzerland), and "TR" is "TR." "E" translates to "ES." "RCH" is now "CL," "SF" translates to "FI," "GH" remains "GH," "RA" becomes "AR," and "DDR" is "DE." I've mapped "NIC" to "NI," and I'm updating "HUN" to "HU". I have confirmed that "ZW" is "ZW", and "TZ" to "TZ." I continue with "KE" to "KE", and confirm that "NIRL" is now "GB". I've decided, for now, that "RB" is "BW." "INT" is now "NA," and "AUT" translates to "AT". Finally, "NZ" remains "NZ," and "BG" is "BG."

Defining the "NL"

I've clarified the "NL" in "Hans Theessink & Blue Groove (NL, USA, UK, GRN)" as the Netherlands. The rest is straightforward: USA is United States, UK is United Kingdom, and GRN is Grenada. The group included singers Terry Evans and Bobby King from the US, along with Colin Hodgkinson from the UK.

Clarifying Country Codes

I've confirmed GRN represents Grenada, and DA is Denmark (DK). I've also mapped several other codes: Vestindien to US Virgin Islands (VI), RCH to Republic of Chile (CL), RA to Republic of Argentina (AR), RB to Republic of Botswana (BW), EAU to Uganda (UG), EUS to Basque Country (ES), DDR to Germany (DE), and INT to NA/International. Finally, I've consolidated UK-related codes to GB, while IRL remains IE (Republic of Ireland).

Mapping Country Codes in R

I'm now focused on the specific R code needed to implement these country code changes. I've compiled a comprehensive list and confirmed the mappings. The aim is to efficiently update the dataset to reflect the correct country codes, with a particular focus on UK-related codes. I'm exploring the best approach in R for a large-scale replacement, paying attention to potential edge cases.

Updating Country Code List

I've expanded and refined the country code mapping, incorporating corrections and validations. I've focused on creating a comprehensive reference, accounting for codes like HUN to HU and EN to GB, alongside more complex examples like EUS to ES and EAU to UG. I'm building a list of ISO alpha-2 codes already present to ensure that those do not get over-ridden. Now, the validation step is completed, and I've focused on how to accomplish the mapping in R.

Implementing in R

I am evaluating the use of `case_match` within `dplyr` or employing a simple, scalable tibble join for country code mapping. I've prepared a mapping table using `tribble`, including various codes and their corresponding ISO alpha-2 codes. I'm focusing on efficient replacement, considering the dataset's scale.

Refining R Implementation

I'm now implementing the `left_join` method in R, incorporating the new code mappings into the `Tonder_grouped` dataset. I've tested the join and subsequent mutate operation to replace

the Nationalitet column with the new ISO alpha-2 codes. I'm focusing on ensuring the new codes are integrated correctly and that the long-lat data will now integrate seamlessly.

Confirming Code Integration

I've confirmed that merging Tonder_grouped and long_lat by "Nationalitet" is the correct approach now, because "Nationalitet" holds the updated ISO alpha-2 codes. The left_join and mutate method is proving effective for the code replacement. Now, I'm noting specific code mappings, highlighting vehicle registration codes and constituent UK countries merging into GB, and confirming that ISO-3/IOC codes map to 2-letter codes.

Validating New Codes

I'm now focusing on mapping remaining codes: AU, MX, HU, AT, EE, GL, and confirming Vestindien to VI and DA to DK. I've successfully linked DDR to DE and SF to FI. I've also reconfirmed that EUS maps to ES, but I am still working on the correct use case of EAU, potentially linking it to Uganda, and RB, which might be Botswana or Bolivia. I'm focusing on the context to differentiate.

Finalizing Code Mappings

I'm now concluding the mapping process. Based on old vehicle codes, I'll map RB to BW (Botswana), but with a note about the potential for Bolivia. I'm confirming that GRN in "Hans Theessink & Blue Groove (NL, USA, UK, GRN)" correctly represents Grenada. I'm focusing on the final presentation: a clear explanation of non-standard codes, a comprehensive mapping list, and the necessary R code (likely tribble and left_join) for seamless implementation in their workflow.

Refining R Implementation

I've decided to refine the R implementation. I'll utilize left_join and a mapping tibble to update the country codes. I'll rename "ISO-ALPHA-2" to "Nationalitet" in the long_lat dataset, ensuring a clean and direct merge using the updated ISO alpha-2 codes generated from the earlier mapping.

Collapse to hide model thoughts

chevron_right

Your dataset contains a mix of different country code standards: **ISO-3 codes** (CAN, USA), **UK constituent countries** (SCO, ENG, WAL), **Danish names** (Vestindien), and old **International Vehicle Registration codes** (D for Germany, F for France, RCH for Chile, EAU for Uganda, etc.).

To successfully merge your Tønder Festival data with your long_lat dataset, you need to map all these variations to the standard **ISO-ALPHA-2** format.

1. Which countries should be changed?

Here is the harmonized mapping from your original codes to the ISO-ALPHA-2 equivalent:

Keep as they are (Already ISO-ALPHA-2):

GB, DK, NL, CZ, SK, FR, NO, IT, BE, CH, TR, GH, ZW, TZ, KE, NZ, BG, AT, EC, FI, FO, SE, GL, ZA, PL.

UK Constituent Countries (Map to GB):

- SCO (Scotland) → GB

- UK (United Kingdom) → **GB**
- ENG / EN (England) → **GB**
- WAL (Wales) → **GB**
- NIRL (Northern Ireland) → **GB**

3-Letter / Custom Codes (Map to ISO-ALPHA-2):

- CAN (Canada) → **CA**
- IRL (Ireland) → **IE**
- USA (United States) → **US**
- AUS (Australia) → **AU**
- MEX (Mexico) → **MX**
- HUN / H (Hungary) → **HU**
- AUT / A (Austria) → **AT**
- EST (Estonia) → **EE**
- GRL (Greenland) → **GL**
- GRN (Grenada) → **GD**
- NIC (Nicaragua) → **NI**

Vehicle Registration / Historic Codes:

- D (Germany / Deutschland) → **DE**
- DDR (East Germany) → **DE**
- F (France) → **FR**
- S (Sweden) → **SE**
- N (Norway) → **NO**
- I (Italy) → **IT**
- E (Spain / España) → **ES**
- P (Portugal) → **PT**
- SF (Finland / Suomi-Finland) → **FI**
- RA (Argentina / República Argentina) → **AR**[\[1\]](#)[\[2\]](#)
- RCH (Chile / República de Chile) → **CL**
- RB (Botswana / Republic of Botswana) → **BW**[\[2\]](#)[\[3\]](#)

- EAU (Uganda / East Africa Uganda) → **UG** (Note: Jon Muq played at Tønder and is from Kampala, Uganda[4])
- DA (Denmark / Danmark) → **DK**

Regional/Other Exceptions:

- Vestindien (US Virgin Islands, formerly Danish West Indies) → **VI**
- EUS (Basque Country / Euskadi) → **ES** (Note: Xabi Aburruzaga played Tønder, representing Basque Country[5][6])
- INT (International) → **NA** (Since it's not a single country)

2. How to do it in R

The cleanest way to harmonize your data in the tidyverse is to create a "lookup table" using `tribble()`, and then join it to your data to replace the old codes.

You can add this block of code right before you merge your data with `long_lat`:

codeR

```
library(tidyverse)
```

1. Create a mapping dictionary for your nationalities

```
country_mapping <- tribble(
  ~old_code, ~iso_2,
  "GB", "GB", "SCO", "GB", "CAN", "CA", "DK", "DK", "IRL", "IE",
  "NL", "NL", "USA", "US", "D", "DE", "F", "FR", "S", "SE",
  "Vestindien", "VI", "CZ", "CZ", "SK", "SK", "FR", "FR", "DA", "DK",
  "H", "HU", "AUS", "AU", "MEX", "MX", "N", "NO", "A", "AT",
  "I", "IT", "UK", "GB", "BE", "BE", "CH", "CH", "TR", "TR",
  "E", "ES", "RCH", "CL", "SF", "FI", "GH", "GH", "RA", "AR",
  "DDR", "DE", "NIC", "NI", "HUN", "HU", "ZW", "ZW", "TZ", "TZ",
  "KE", "KE", "NIRL", "GB", "RB", "BW", "INT", NA_character_, "AUT", "AT",
  "NZ", "NZ", "BG", "BG", "ENG", "GB", "AT", "AT", "EC", "EC",
  "P", "PT", "FI", "FI", "FO", "FO", "NO", "NO", "SE", "SE",
  "GL", "GL", "EST", "EE", "GRL", "GL", "ZA", "ZA", "WAL", "GB",
  "IT", "IT", "EN", "GB", "PL", "PL", "EAU", "UG", "GRN", "GD",
```

```
"EUS", "ES"  
)
```

2. Update Tonder_grouped with the new ISO-ALPHA-2 codes

```
Tonder_grouped_clean <- Tonder_grouped %>%
```

```
# Join our lookup table
```

```
left_join(country_mapping, by = c("Nationalitet" = "old_code")) %>%
```

```
# Replace the old column with the new ISO codes
```

```
mutate(Nationalitet = iso_2) %>%
```

```
# Drop the extra lookup column
```

```
select(-iso_2) %>%
```

```
# Remove rows that evaluated to NA (like "INT") if you don't want them plotted
```

```
drop_na(Nationalitet)
```

3. Now merge with your long_lat dataset

```
# (Ensure you've run: colnames(long_lat)[3] <- "Nationalitet" first as per your script)
```

```
Tonder_long_lat <- merge(Tonder_grouped_clean, long_lat, by = "Nationalitet") %>%
```

```
arrange(År)
```

```
head(Tonder_long_lat)
```

By standardizing to ISO-ALPHA-2 *before* the merge, the data will seamlessly align with your coordinate dataset and you'll easily be able to plot the artists dynamically on a map.

Sourceshelp

1. celticlifeintl.com
2. iej.org.za
3. theafterword.co.uk
4. ritzau.dk
5. politikenplus.dk
6. resonator.dk

Google Search Suggestions

Display of Search Suggestions is required when using Grounding with Google Search. [Learn more](#)

-
-
-
-
-
-
-

2m17.7s

info

Google AI models may make mistakes, so double-check outputs.

warning

Turn Drive on for future conversations

Your work is currently not being saved and will be lost, we recommend enabling Google Drive, you will always have the option to turn on Temporary chat for conversations you do not want to save to your chat history.

Use Arrow Up and Arrow Down to select a turn, Enter to jump to it, and Escape to return to the chat.

googleGrounding with Google Sear